Build-Along Security Lab

A Step-by-Step Guide to Standing Up a Security Practice Testbed

Weekend build • single workstation • free / no-cost tier

Introduction

Security best practices such as least privilege, patch management, multi-factor authentication, and vulnerability scanning only become real once you have configured them with your own hands, watched them fail in unexpected ways, and adjusted them until they behave the way the documentation promised. Production systems are the wrong place to discover that an MFA rollout broke a service account, that a just-in-time workflow left an orphaned scheduled task, or that a new vulnerability scanner overwhelmed a legacy system.

This guide walks through the construction of a self-contained laboratory environment that absorbs those lessons at zero operational cost. Every component is either free or has a no-cost tier sufficient for learning, and the entire build is designed to fit on a single workstation with 16 GB of RAM.

What You Will Build

By the end of this guide you will have:

- A small Active Directory domain with two member servers and two workstations
- A Linux host running a vulnerable web application for scanning practice
- A free-tier identity provider enforcing MFA, including a phishing-resistant option
- A just-in-time access workflow that grants and revokes admin rights on a schedule
- A vulnerability scanner producing real findings against the lab estate
- A patch management workflow that closes the loop from scan to remediation

Prerequisites

Confirm that your workstation meets the recommended specifications below before beginning. The minimum column will work for a smaller subset of the lab but may require shutting down VMs you are not actively using.

Component	Recommended	Minimum
Host OS	Windows 11 Pro, macOS 13+, or Ubuntu 22.04+	Any 64-bit OS supporting virtualization

Component	Recommended	Minimum
CPU	8 cores with VT-x / AMD-V	4 cores
RAM	32 GB	16 GB
Storage	250 GB SSD	150 GB SSD
Hypervisor	VirtualBox 7.x (free) or VMware Workstation Player	Either of the above
Network	Two host-only networks (LAB-INTERNAL and LAB-DMZ)	Single host-only network

Lab Topology Overview

The lab consists of five virtual machines spread across two host-only networks. The internal network (10.10.10.0/24) hosts the Active Directory domain and management systems, while the DMZ network (10.10.20.0/24) isolates the deliberately vulnerable web application.

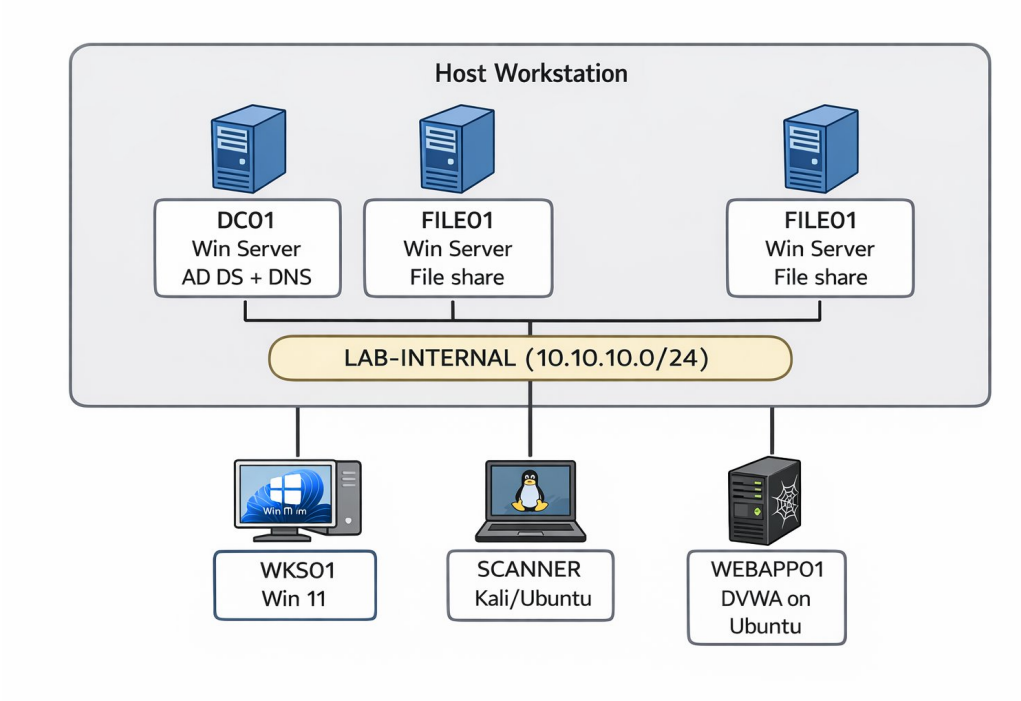

Figure 1 – Lab Topology

VM Name	Operating System	Role
DC01	Windows Server 2022	Domain controller for lab.local; AD DS and DNS

VM Name	Operating System	Role
	(eval)	
FILE01	Windows Server 2022 (eval)	Domain-joined file server hosting \\FILE01\SensitiveData
WKS01	Windows 11 (eval)	Domain-joined standard user endpoint
WEBAPP01	Ubuntu 22.04	Runs Damn Vulnerable Web Application (DVWA) in Docker
SCANNER	Ubuntu 22.04	Hosts Nessus Essentials (free for up to 16 IPs)

Table 1 – System List

Step 1 — Stand Up the Core Infrastructure

Estimated time: 90 minutes

Install VirtualBox and create the two host-only networks that the lab depends on. Then build each base VM using the evaluation or free images listed below.

1.1 Create host-only networks

- LAB-INTERNAL → 10.10.10.0/24
- LAB-DMZ → 10.10.20.0/24

1.2 Build the virtual machines

1. DC01 — Windows Server 2022 evaluation ISO (180-day trial). Install AD DS, promote to domain controller for lab.local, and configure DNS.
2. FILE01 — Windows Server 2022, domain-joined, hosting a file share at \\FILE01\SensitiveData.
3. WKS01 — Windows 11 evaluation, domain-joined, representing a standard user endpoint.
4. WEBAPP01 — Ubuntu 22.04 running Damn Vulnerable Web Application (DVWA) in Docker. One command starts it:

```
docker run --rm -it -p 80:80 vulnerables/web-dvwa
```

5. SCANNER — Ubuntu 22.04 with Nessus Essentials installed (free for up to 16 IPs, sufficient for this lab).

1.3 Create lab user accounts

In Active Directory, create three user accounts that will be used throughout the remaining steps:

- alice — standard user
- bob — developer
- admin-jit — target account for the just-in-time workflow

Step 2 — Implement Least Privilege and RBAC

Estimated time: 45 minutes

In Active Directory, create security groups that map to job functions rather than to individual people. Group-based assignment is the foundation of every RBAC model and the prerequisite for the just-in-time workflow built later.

Group	Members	Permissions
GG-FileShare-HR-Read	alice	Read on \\FILE01\HR
GG-FileShare-Dev-Write	bob	Read/Write on \\FILE01\Dev
GG-ServerAdmins-Tier1	(empty by default)	Local admin on FILE01

Table 2 - Permissions

Validate the configuration

Configure the file shares with NTFS permissions tied to these groups, then test the boundaries:

- Log in as alice and confirm that she cannot write to the Dev share.
- Log in as bob and confirm that he cannot read the HR share.

This establishes the baseline you will later elevate against.

Step 3 — Deploy MFA with a Free Identity Provider

Estimated time: 60 minutes

Create a free Microsoft Entra ID tenant (formerly Azure AD free tier) or use Okta's developer free tier. Both support FIDO2, TOTP, and conditional access policies at no cost for lab-scale use. The walkthrough below uses Entra ID.

3.1 Configure the tenant and policies

6. Create the tenant and sync or recreate alice, bob, and admin-jit as cloud users.
7. Register a test application — any SaaS application or an internal app published through Application Proxy will do.

8. Create two Conditional Access policies:
    - CA01 — Require MFA for all users accessing the test app.
    - CA02 — Require phishing-resistant MFA (FIDO2) for admin-jit.
9. Enroll a TOTP authenticator (Microsoft Authenticator or any RFC 6238 app) for alice and bob.
10. Register a hardware key (YubiKey 5, SoloKey, or equivalent) for admin-jit. If a hardware key is unavailable, use Windows Hello for Business on WKS01 as the phishing-resistant factor.

3.2 Test the policies

Test each policy by signing in as the relevant user. Then intentionally trigger an MFA fatigue scenario by attempting multiple sign-ins for alice and observing how the authenticator behaves. This is where number-matching, enabled by default in Entra since 2023, proves its value.

Step 4 — Build a Just-in-Time Access Workflow

Estimated time: 60 minutes

Entra ID's free tier does not include Privileged Identity Management, so the lab uses a lightweight JIT workflow built from PowerShell and scheduled tasks. This is exactly the kind of pattern a resource-constrained team might start with before adopting a commercial PAM platform.

4.1 Create the grant script

On DC01, save the following script as C:\Scripts\Grant-JitAdmin.ps1:

```
param(
   [Parameter(Mandatory)][string]$SamAccountName,
   [Parameter(Mandatory)][int]$DurationMinutes,
   [Parameter(Mandatory)][string]$TicketId
)

$user   = Get-ADUser -Identity $SamAccountName
$group  = "GG-ServerAdmins-Tier1"
$expiry = (Get-Date).AddMinutes($DurationMinutes)

Add-ADGroupMember -Identity $group -Members $user
Write-EventLog -LogName Security -Source "JIT" -EventId 4001 `
   -Message "JIT grant: $SamAccountName added to $group until $expiry for ticket $TicketId"

$action = New-ScheduledTaskAction -Execute "PowerShell.exe" `
   -Argument "-Command Remove-ADGroupMember -Identity '$group' -Members
'$SamAccountName' -Confirm:\$false"
```

```
$trigger = New-ScheduledTaskTrigger -Once -At $expiry
Register-ScheduledTask -TaskName "Revoke-JIT-$SamAccountName-$(Get-Random)" `
  -Action $action -Trigger $trigger -User "SYSTEM" -RunLevel Highest
```

4.2 Invoke and validate the workflow

Invoke the script to grant admin-jit membership in the server admin group for thirty minutes:

```
.\Grant-JitAdmin.ps1 -SamAccountName admin-jit -DurationMinutes 30 -TicketId CHG-001
```

Verify the grant on FILE01, perform an administrative action, then wait for the scheduled task to fire and confirm that the membership has been removed. You have just built the smallest possible working JIT system. Scaling this to production is the work that PAM platforms such as CyberArk, Delinea, and Azure PIM exist to do, but the underlying concept is identical.

Step 5 — Run a Vulnerability Scan

Estimated time: 45 minutes

From the SCANNER VM, configure Nessus Essentials with two scan policies:

- Policy A — Unauthenticated network scan targeting 10.10.10.0/24 and 10.10.20.0/24.
- Policy B — Credentialed scan using a dedicated svc-scanner AD account with read-only local access on member servers, plus SSH credentials for WEBAPP01.

Run both scans and compare the findings. Policy A will surface a handful of network-visible issues; Policy B will typically produce five to ten times as many findings because it can enumerate installed packages and check patch levels directly. This is the credentialed-vs-unauthenticated gap made tangible.

DVWA on WEBAPP01 will produce intentional findings across the OWASP Top 10. Use them to practice classification: sort by CVSS, then re-sort using CISA's Known Exploited Vulnerabilities (KEV) catalog as a filter, and notice how the prioritization shifts.

Step 6 — Close the Remediation Loop

Estimated time: 30 minutes

Provision a free Jira Cloud instance (free for up to 10 users) or a self-hosted GitLab instance. Configure Nessus to export findings as CSV, then write a small Python script to ingest the CSV and create tickets tagged by severity and asset.

Reference SLA mapping

Severity	Critical	High	Medium / Low
SLA (days)	3	14	30 / 90

Table 3 - SLA

Sample ingestion script

```
import csv, requests, os
from requests.auth import HTTPBasicAuth

JIRA_URL = "https://your-domain.atlassian.net"
AUTH = HTTPBasicAuth(os.environ["JIRA_EMAIL"], os.environ["JIRA_TOKEN"])

SLA = {"Critical": 3, "High": 14, "Medium": 30, "Low": 90}

with open("nessus_export.csv") as f:
    for row in csv.DictReader(f):
        severity = row["Risk"]
        if severity not in SLA:
            continue
        payload = {
            "fields": {
                "project": {"key": "SEC"},
                "summary": f"[{severity}] {row['Name']} on {row['Host']}",
                "description": row["Description"],
                "issuetype": {"name": "Task"},
                "labels": [f"sla-{SLA[severity]}d", f"cvss-{row['CVSS v3.0 Base Score']}"]
            }
        }
        requests.post(f"{JIRA_URL}/rest/api/3/issue", json=payload, auth=AUTH)
```

Step 7 — Exercise Patch Validation

Estimated time: 45 minutes

Deliberately hold back a Windows update on WKS01, run a Nessus scan to confirm that the missing patch appears as a finding, then apply the update and rescan to confirm that the finding closes. Repeat the exercise with the DVWA container: pull the latest image, rescan, and observe which web findings remain regardless. DVWA is vulnerable by design — a useful reminder that patching and secure coding are different disciplines.

Optional Extensions

Once the core lab is running, each of the following extensions deepens a specific practice:

- Replace the PowerShell JIT script with Azure PIM (requires an Entra P2 trial) or an open-source PAM such as Teleport Community Edition.
- Swap Nessus Essentials for OpenVAS to compare detection logic across two engines on the same targets.
- Add a second host network segment and Windows Defender Firewall rules to practice network segmentation.
- Deploy a free tier of Microsoft Defender for Cloud against an Azure free account to compare cloud-native scanning against Nessus on equivalent workloads.
- Integrate a phishing simulation using GoPhish (free, self-hosted) and measure click-through against your trained users.

Cost and Cleanup

The entire lab as described uses free evaluation licenses and free-tier SaaS, with optional hardware investment for a FIDO2 key (approximately $25 to $50). Microsoft Server evaluation expires after 180 days and Windows 11 evaluation after 90 days. When you are finished, exporting each VM as an OVA file before expiration lets you rebuild the lab later in minutes rather than hours.

What You Walk Away With

Running through these seven steps produces more than a working lab. It produces a reference environment that you can return to every time a new practice needs testing — a place to try conditional access policy changes before pushing them to production, a sandbox for validating patches on representative system configurations, and a teaching environment for bringing new team members up to speed on the controls covered in this guide.

The real value is not the specific lab topology; it is the habit of having somewhere safe to break things and learn from the breakage, so that the production environment never has to be that place.